

From Source Linkage to Explanation Fidelity: An Executable Audit of Enterprise KPI Knowledge Publication

Meshaal Obaid Mouawad^{a,*}

^aDepartment of Electrical and Computer Engineering, Mississippi State University, Mississippi State, 39762, MS, USA

ARTICLE INFO

Keywords:

software documentation
program comprehension
provenance
traceability
key performance indicators
explanation fidelity
knowledge engineering

ABSTRACT

Context: Provenance can remain intact while software-derived documentation misstates the role, behavior, disagreement, or status of its linked evidence. **Objective:** We define a literature-grounded publication checklist with four auditable obligations—source-role accuracy, condition disclosure, representation-disagreement preservation, and review-authority consistency—and test both its defect-localization value in LEAP and its procedural transferability to an independently developed documentation generator. **Method:** Deterministic rules audit a current frozen 28-dossier LEAP build, an earlier 18-dossier LEAP build, and two additional LEAP input configurations. A separate external pilot applies the checklist to four official FastAPI sources and generated OpenAPI 3.1 descriptions. Before the final rerun, the repaired checker is exercised against 18 predeclared rule fixtures and 14 artifact-derived controls comprising five unchanged baselines, eight substitutions of extracted inputs, and one document-level HTML mutation reparsed through the publication adapter. **Results:** All 32 checker-validation outcomes match their expected labels, with zero misses on defect controls and zero false alarms on benign controls. The LEAP audit then localizes three generator defects—lineage typing, guard-to-safeguard binding, and unconditional certification language—plus one correctly operating conflict-routing mechanism. Within-system persistence/configuration results remain bounded as such. In the external FastAPI pilot, all seven directly applicable source-role and condition checks pass and nine checklist cells are not applicable; two separately labeled deprecation-status analogues also pass. **Conclusion:** The contribution is a checker-validated, reproducible publication-audit procedure with explicit applicability and adaptation rules, plus localized evidence of three LEAP generator defects. The external pilot demonstrates transfer of the role/condition audit procedure beyond LEAP without claiming external prevalence or universal applicability of all four obligations.

1. Introduction

A source link can be correct while the explanation attached to it is wrong. A documentation system may point to the right function yet describe a comment as executable output, omit a condition that qualifies behavior, collapse a known representation disagreement, or display unresolved review state beside language that implies certification. These are publication failures that provenance alone does not settle.

Prior work already supplies the component ideas: behavioral completeness and code–text consistency (Eghbali et al., 2026; Wen et al., 2019; Xu et al., 2024), inconsistency management (Nuseibeh et al., 2001), evidence and assertion roles (Brush et al., 2016; Giglio et al., 2019), provenance and validity (Missier et al., 2013; World Wide Web Consortium, 2013), operational lineage and governance (Psallidas et al., 2023; Sadiq et al., 2026), and emerging source-bound projections (Gao, 2026; de Macedo and da Costa, 2026). None of those foundations is claimed as new here.

This paper asks a narrower engineering question: once source evidence, declared representations, reader-facing explanation, and status metadata coexist on a publication surface, what should an audit check? We synthesize four obligations from the established literature: describe source elements according to their actual role; disclose conditions that materially qualify the published behavior; retain explicit

representation disagreement until it is resolved; and align publication language with recorded review authority. We use *explanation fidelity* as shorthand for this publication checklist.

The checklist does not claim that the four obligations are complete, minimal, mutually exclusive, or statistically independent, and it does not claim an interaction effect among them. Its purpose is practical defect localization with explicit applicability gates. The central methodological question is whether that procedure can be instantiated outside the system from which it was developed.

We first audit LEAP (Literate Programming for Automated KPI Extraction), which publishes source-linked KPI dossiers through an *Interactive Bluebook*. We then conduct a small external transferability pilot on FastAPI, an independently developed framework that generates OpenAPI descriptions from annotated Python endpoints. The external pilot is intentionally narrow: it tests whether source-role and condition checks transfer directly, records not-applicable outcomes where the external corpus lacks unresolved representation conflict or human review state, and separately tests lifecycle-status propagation as an analogue rather than relabeling it as review authority. The paper contributes:

1. a literature-grounded, executable four-obligation publication checklist with explicit applicability rules;
2. a deterministic complete-corpus audit and root-cause localization of three systematic LEAP generator defects plus one correctly operating conflict-routing mechanism;

*Corresponding author

 mm4922@msstate.edu (M.O. Mouawad)

ORCID(s): 0000-0002-1152-8324 (M.O. Mouawad)

3. temporal and input-configuration persistence checks within LEAP, reported explicitly as within-system evidence rather than independent replication; and
4. an external FastAPI/OpenAPI pilot that tests which parts of the audit procedure transfer without LEAP-specific assumptions and which remain not applicable or require an explicit analogue.

2. Related Work and Derivation of the Publication Contract

2.1. Behavioral fidelity and source roles

Software documentation remains difficult to create and maintain (Aghajani et al., 2019). Traceability can locate the related artifact without establishing that the explanation is semantically correct (Mäder and Gotel, 2012). Code-comment inconsistency studies make this distinction explicit (Wen et al., 2019; Xu et al., 2024). Eghbali et al. formalize code-documentation equivalence around accurate and complete description of program behavior (Eghbali et al., 2026). Closely related API-documentation work by Zhou et al. compares code with parameter-constraint and exception directives to detect documentation defects (Zhou et al., 2017). Condition disclosure is therefore not claimed as a new problem or a new API-directive detector here. The present checklist uses that established concern together with source-role, disagreement, and authority checks at a multi-representation publication boundary.

2.2. Disagreement, evidence roles, and authority

Nuseibeh et al. show that inconsistency can be retained and managed rather than silently eliminated (Nuseibeh et al., 2001). SEPIO represents claims, evidence lines, data, methods, tools, agents, and supporting or refuting relations (Brush et al., 2016); ECO distinguishes evidence and assertion-method types (Giglio et al., 2019). These precedents motivate retaining explicit representation disagreement and distinguishing evidence state from the authority conveyed by surrounding prose.

PROV supplies interoperable entity–activity–agent provenance and formal validity constraints (Missier et al., 2013; World Wide Web Consortium, 2013). OneProvenance reconstructs dynamic query lineage from database events (Psallidas et al., 2023). ESDPKI adds semantic provenance, validation-gated ingestion, publication, governance, and access controls in an enterprise infrastructure (Sadiq et al., 2026). These systems establish that provenance, validation, and governance already have mature formalisms; the present work audits how those states are communicated on a software publication surface.

2.3. Source-bound software knowledge

ECK, an arXiv preprint, defines source-bound executable knowledge with evidence, provenance, validation state, freshness, and downstream projections; its evaluation also includes projection-fidelity studies (Gao, 2026). Reversa, also an arXiv preprint, preserves traceability, confidence, and explicit

validation gaps in reconstructed specifications (de Macedo and da Costa, 2026). The present work does not claim projection fidelity or source-bound validation as new. Its narrower target is an executable audit of publication-specific source roles, condition disclosure, unresolved representation disagreement, and review/status communication on a multi-representation record.

KPI semantics provide the domain context rather than the novelty basis. Semantic KPI work formalizes indicator definitions and derivations (Maté et al., 2017; Roldán-García et al., 2021; Diamantini et al., 2025), and AFFECTK makes traceability and consistency explicit during KPI evolution (Domínguez et al., 2025). The publication contract is applied here where such representations meet executable source and review state.

3. Research Questions and Method

The study asks:

- RQ1** Which systematic publication defect classes and successful mechanisms are exposed by the checklist in the current complete LEAP dossier build, and what generator logic explains them?
- RQ2** Which manifestations persist across an earlier LEAP build and alternative LEAP input configurations, and which do not?
- RQ3** Which parts of the publication-audit procedure transfer to an independently developed generator, and what applicability or adaptation boundaries are exposed by that transfer?

3.1. LEAP artifacts and within-system persistence design

The primary LEAP corpus contains all 28 rendered dossier HTML pages from the frozen sample-project publication artifact. A public artifact copy at commit 5e697b5b (full identity in the supplement) matches all 28 submitted pages byte-for-byte by SHA-256. It contains 9 *Validated* and 19 *Needs Review* dossiers. The 19 unresolved pages define eligibility for the authority-language rule; they are not a quality-prevalence estimate.

A separately archived earlier LEAP build contains 18 rendered dossier HTML pages, including 6 *Validated* and 12 *Needs Review* dossiers. It is a prior state of the same project and shares substantial sample-project content with the current build. It is therefore used only to test temporal persistence of named manifestations, not as an independent replication corpus.

Two further frozen LEAP input configurations use the same generator: a 38-file scenario-fixture scope producing 29 dossier publication sources and an 8-file challenge scope producing 8. These runs test input sensitivity. They are not pooled with the rendered-HTML corpora and are not treated as independent systems or population samples.

Table 1

Literature basis and operational rule for each checklist obligation. The table defines an audit synthesis, not new underlying concepts.

Obligation	Established basis	Operational rule used in LEAP
Source-role accuracy	Code-text consistency and documentation equivalence (Wen et al., 2019; Xu et al., 2024; Eghbali et al., 2026)	Flag a published explanation that assigns a source element the wrong syntactic/semantic role.
Condition disclosure	Completeness of behavioral/API directives (Eghbali et al., 2026; Zhou et al., 2017)	For explicit zero/non-positive denominator guards, require reader-facing text to preserve the guarded variable and condition semantics.
Disagreement preservation	Inconsistency management and conflicting evidence (Nuseibeh et al., 2001; Brush et al., 2016)	When source evidence explicitly records conflicting representations, require the conflict to remain visible in review routing.
Review-authority consistency	Evidence/assertion roles, provenance validity, and validation-governance separation (Giglio et al., 2019; World Wide Web Consortium, 2013; Sadiq et al., 2026)	When a dossier is unresolved, flag publication wording that implies completed certification.

The LEAP unit of analysis is a generated dossier. The audit does not execute KPI functions, infer approved business meaning, or estimate source-candidate precision or recall.

3.2. External transferability pilot

To test whether the audit procedure can be instantiated outside LEAP, we use four official FastAPI tutorial source files from the public `fastapi/fastapi` repository at commit `33d411dbc3236275dd64d200bfe18d5d60a49b2e`. The files cover a constrained query parameter, constrained path/query parameters, constrained request-body fields, and a deprecated path operation. Their Git blob identities are retained in the supplement. We execute those examples with FastAPI 0.128.2 and Pydantic 2.13.4 and archive the generated OpenAPI 3.1 descriptions and runtime validation outcomes (FastAPI Project, 2026; OpenAPI Initiative, 2026).

The selection is purposive and capability-oriented, not a prevalence sample: the examples were chosen to exercise parameter/request-body roles, published validation constraints, and explicit deprecation metadata. All four checklist obligations are evaluated with applicability gates. Source-role accuracy maps directly to OpenAPI parameter location or request-body placement. Condition disclosure maps to declarative validation constraints (for example minimum, maximum, pattern, and field constraints); this is a stated adaptation because FastAPI publishes validation metadata rather than arbitrary internal branch guards. Representation-disagreement preservation remains not applicable when the example contains no unresolved explicit conflict. Review-authority consistency remains not applicable because these examples contain no human review state. Separately, where `deprecated=True` is present, we test lifecycle-status propagation as an analogue; it is not counted as a direct pass of the review-authority obligation.

3.3. Deterministic operational rules

All outcome coding in this revision is deterministic; no human adjudication contributes to the reported counts.

1. **Source-role accuracy.** A failure is recorded when a comment or docstring is assigned an executable role, or when a SQL CAST type is labeled as the output alias instead of the actual alias. The rule scans the published developer-lineage pairs.
2. **Executable-guard disclosure.** Eligibility is restricted to explicit zero or non-positive denominator guards expressed as `NULLIF(variable,0)` or an `if-family` comparison to zero. A guard is preserved only when reader-facing safeguard text explicitly communicates zero/non-positive handling and the guarded variable matches the denominator displayed in the formal formula. A non-positive `<=0` guard is not credited by zero-only wording. Threshold conditions unrelated to denominator protection are outside this dimension.
3. **Representation-disagreement preservation.** Eligibility is restricted to pages whose retained source explicitly carries a conflict, mismatch, or inversion marker. Preservation requires *Needs Review* and a Review Queue.
4. **Review-authority consistency.** Eligibility is restricted to *Needs Review* dossiers. A failure occurs when the same page contains bounded certification-oriented wording such as “Certified metric dossier”, “certified calculation expression”, or “Print Certified Dossier”.

Because these are executable rules rather than coder judgments, inter-rater reliability is not an outcome of this analysis. The complete guard table and all line-level role findings are emitted by the scripts for inspection. The rule definitions themselves are grounded in the literature summarized in Table 1; they are not claimed to be an exhaustive taxonomy of explanation quality.

3.4. Rule-development chronology and checker validation

The condition rule was developed after earlier inspection of the LEAP artifact. An earlier research version used a

post-hoc 11-row manual condition adjudication; that table is not part of the present analysis. The retained R3 protocol records its replacement by deterministic rules before the later corpus reruns. Subsequent negative-control testing of the R4 implementation exposed two checker faults: one external query-role cell was assigned *Pass* without comparing the generated role/name to the source, and the LEAP guard checker could accept an empty denominator or contradictory safeguard wording. Both faults were repaired before the results reported here were regenerated.

The released checker is validated in two execution layers against predeclared control-oracle tables stored separately from checker output. First, 18 rule fixtures exercise correct and substituted source roles, correct/missing/wrong denominator binding, contradictory or disabled safeguard prose, and zero-only loss of non-positive semantics, preserved/suppressed conflict routing, certification beside unresolved status, benign controls, and N/A behavior. Second, 14 artifact-derived controls comprise five unchanged baselines, eight substitutions of extracted inputs, and one document-level HTML mutation reparsed through the same publication adapter used for the archived LEAP page. Across the 32 oracle-labeled outcomes, all labels match; among the 30 binary Pass/Fail controls there are zero misses on deliberately introduced defects and zero false alarms on benign controls. These controls validate the released predicates on specified edge cases; they are not a population estimate of checker sensitivity or specificity.

3.5. Verification and root-cause procedure

The primary audit verifies corpus size, SHA-256 identity, status counts, lineage pairs, guard instances, conflict routing, and certification wording. The primary, historical, robustness, and external scripts import the same released rule library for denominator binding, safeguard semantics, conflict routing, authority consistency, and source-role comparison where applicable. A separate historical-build script applies those rules to the archived second corpus, and two additional scripts audit the scenario and challenge configurations. The primary guard inventory contains ten eligible guards and is produced directly by the scanner rather than by a manually adjudicated table. The release verifier requires the control-oracle comparison and both checker-validation execution layers to pass before accepting the corpus summaries.

After classifying page manifestations, we inspect the frozen generator source to determine whether repeated manifestations share one implementation cause. Root-cause evidence is reported only when a single source mechanism explains the repeated page behavior. The supplement includes the exact generator excerpts and source hashes used for this trace.

4. Results

4.1. Three LEAP defect classes and one working mechanism

The current 28-dossier LEAP build reduces to three systematic generator defects plus one correctly operating

conflict-routing mechanism. Table 2 reports manifestations separately from defect classes; page and guard counts are not treated as independent defects or prevalence estimates.

The two alternative LEAP configurations do not reproduce all three defect classes. The 29-dossier scenario build shows one lineage defect and the authority-language defect on all 11 unresolved dossiers, while its one eligible zero guard is communicated correctly. The 8-dossier challenge build likewise shows one lineage defect and the authority-language defect on all six unresolved dossiers, while its one eligible zero guard is communicated correctly. Thus input variation reproduces two defect classes but not the guard-binding defect. Generated conflict flags route correctly in both configurations.

4.2. Root-cause localization

The repeated LEAP manifestations map to three generator mechanisms. First, the developer-lineage classifier tests calculation/formula keywords before comment typing, does not recognize the ABAP * comment prefix, and applies a generic SQL AS pattern that can select a CAST type rather than the final alias. This classifier explains the source-role manifestations.

Second, safeguard generation recognizes NULLIF and selected equality-to-zero patterns and emits generic “zero denominator” language. It does not bind the guarded variable to the denominator shown in the formal formula and does not preserve ≤ 0 semantics. This explains the current guard failures while allowing correctly aligned NULLIF cases to pass.

Third, certification-oriented strings are emitted independently of review state in both generator prose and the page template. That mechanism explains why every unresolved page inherits certification language. In contrast, explicit conflicts are intentionally routed to *Needs Review* and the Review Queue, explaining the successful conflict-preservation result.

4.3. Checker validation

The repaired rule implementation matches all 18 predeclared rule fixtures and all 14 artifact-derived controls. The controls include the two previously failing cases: changing the external query parameter from *query/item-query* to a header or wrong name now fails the source-role check, and removing the displayed denominator, replacing safeguard prose with “No zero handling is provided.”, stating that the zero guard is disabled, or reparsing a page whose explanatory sentence is replaced by a second “Zero-Division Guard:” heading now fails the guard check. Deliberately suppressing a Review Queue entry from an explicit conflict and placing certification wording beside unresolved status are also detected. Across the 30 binary controls, the checker produces zero misses on defect controls and zero false alarms on benign controls; two additional fixtures verify N/A behavior. These are controlled implementation-validation results, not population-level accuracy estimates.

Table 2

Within-LEAP persistence results. The historical build is an earlier state of the same project, not an independent replication corpus.

Class / mechanism	Current 28-dossier build	Earlier 18-dossier build	Interpretation
Lineage typing defect	4 dossiers; 12 manifestations	Same 4 dossiers; same 12 manifestations	Persistence of the same classifier defect across project states; not independent replication
Guard-to-safeguard binding defect	5 failures / 10 eligible guards	4 failures / 9 eligible guards	Same generator weakness, with one changed eligible outcome across states
Authority-language template defect	19 / 19 unresolved dossiers	12 / 12 unresolved dossiers	Same unconditional template behavior on each build's unresolved pages
Conflict routing mechanism	Same 6 explicit-conflict dossiers, 6 / 6 preserved	Same 6 dossiers, 6 / 6 preserved	Persistence of the intended routing behavior on shared conflict cases

Table 3

External FastAPI/OpenAPI transferability pilot. "Status analogue" is lifecycle/deprecation propagation, not a direct review-authority result.

Official FastAPI example	Source role	Condition disclosure	Disagreement	Review authority / status analogue
Constrained query parameter	Pass	Pass	N/A	N/A; status analogue Pass
Constrained path/query parameters	Pass	Pass	N/A	N/A
Constrained request-body fields	Pass	Pass	N/A	N/A
Deprecated path operation	Pass	N/A	N/A	N/A; status analogue Pass

4.4. External FastAPI/OpenAPI transferability pilot

Table 3 reports the external pilot after checker repair. The role checks now compare source-declared parameter/request-body roles and names explicitly with generated OpenAPI locations/names; condition checks compare source-declared constraints with generated schema constraints and runtime enforcement. Across the four official FastAPI examples, seven directly applicable source-role or condition checks all pass; nine of the 16 example-by-obligation cells are correctly classified not applicable. Two negative mutations of the query role/name are rejected. In addition, two separately labeled lifecycle-status analogue checks pass: source-level deprecated=True is propagated to OpenAPI deprecated=true. Invalid values for the three constrained examples return HTTP 422, confirming that the constraints represented in the generated description are enforced by the external framework.

The external result is intentionally bounded. It demonstrates that the audit procedure can be applied to independently authored source and an independently developed publication generator for role and condition checks, and that its applicability gate can return N/A rather than inventing a conflict or review-state analogue. It does not establish documentation-defect prevalence in FastAPI, and it does not externally validate the exact LEAP review-authority or disagreement rules because the selected external examples do not expose those states.

5. Detailed Case Analysis

5.1. Ethylene Production Yield

The Yield source contains a percentage calculation and a non-positive-input guard: The dossier displays the positive-

```

1 def calculate_ethylene_yield_pct(ethylene_produced_tons,
2                                 feedstock_input_tons):
3     """Calculates ethylene production yield as a percentage."""
4     if feedstock_input_tons <= 0:
5         return 0.0
6     return (ethylene_produced_tons / feedstock_input_tons) * 100.0

```

Listing 1: Selected Yield source retained in the dossier.

input ratio as its Formal Formula. Its reader-facing safeguards do not state the non-positive-input behavior. For the docstring shown in Listing 1, the generated lineage prose is "Output alias: maps this expression to a in the result set." The source is a function docstring, not an output alias. The case therefore instantiates the lineage-typing and guard-binding defect classes.

5.2. Gross Margin Percentage

The Gross Margin dossier retains the declared formula

$$D = 100 \frac{R - C}{R}, \quad (1)$$

while the implementation, after guarding zero cost, returns

$$I = 100 \frac{R}{C}. \quad (2)$$

For $R > 0$ and $C > 0$,

$$I - D = 100 \left(\frac{R}{C} + \frac{C}{R} - 1 \right) \quad (3)$$

$$= 100 \left(1 + \frac{(R - C)^2}{RC} \right) \geq 100. \quad (4)$$

Thus the implemented expression exceeds the declaration by at least 100 percentage points throughout the strictly positive domain, with equality in this lower bound when $R = C$. At positive revenue and zero cost, the declaration evaluates to 100 while the implementation guard returns zero; at zero revenue and positive cost, the declaration is undefined while the implementation returns zero.

The dossier correctly routes the explicit disagreement through *Needs Review* and a Review Queue. Its safeguard text nevertheless describes a generic zero denominator even though the declared and executable representations use different denominators, and certification-oriented language appears alongside unresolved status. One page therefore passes disagreement preservation while failing guard binding and authority consistency.

6. Discussion

6.1. Answers to the research questions

RQ1. The current complete LEAP build exposes three systematic generator defects—lineage typing, guard-to-safeguard binding, and unconditional authority language—plus a conflict-routing mechanism that operates as intended. Root-cause tracing maps the repeated page manifestations to those generator mechanisms rather than treating each occurrence as an independent empirical event.

RQ2. The earlier LEAP build shows persistence, not independent replication. The same four dossiers carry the same 12 lineage manifestations, and the same six explicit-conflict dossiers are routed correctly in both builds. Guard outcomes differ by one eligible case (5/10 failures current; 4/9 earlier), while the unconditional authority template affects every unresolved page in each build. The two alternative configurations reproduce the lineage and authority defects but not the guard-binding defect. The within-system evidence therefore separates persistent generator behavior from input-sensitive manifestations.

RQ3. After repairing and validating the checker against 30 binary controls plus two N/A applicability fixtures, the external FastAPI/OpenAPI pilot demonstrates partial procedural transfer. Source-role and declarative-condition checks can be instantiated directly on an independently developed generator, producing seven applicable checks and seven passes. The exact disagreement and review-authority obligations correctly return N/A because the selected external examples contain neither unresolved representation conflicts nor human review state. A separately labeled deprecation-status

analogue also propagates correctly. The method’s transferable element is therefore the audit procedure—explicit evidence roles, applicability gates, comparison rules, checker controls, and reproducible outputs—not a claim that every semantic obligation exists unchanged in every documentation system.

6.2. Engineering implications

The LEAP findings lead to three bounded fixes. Source-line explanations should type the statement before generating role-specific prose. Safeguards should bind each condition to the variable and representation it governs. Certification-oriented wording should be conditional on review state. Conflict routing should be retained because it keeps explicit disagreement visible rather than silently selecting one representation.

The audit also separates two verification levels. Page-level manifestations show where communication is wrong; root-cause tracing identifies the generator mechanism to repair. This prevents a repeated template defect from being misreported as many independent scientific findings while retaining the practical value of knowing how broadly the defect propagates.

The external pilot adds a separate lesson: portability requires explicit adapters. OpenAPI has standardized parameter locations, schemas, and deprecation metadata ([OpenAPI Initiative, 2026](#)); these make role and declarative-condition checks straightforward but do not create a human review state or unresolved representation conflict. A transferable audit should therefore preserve N/A as a first-class result rather than forcing every source system into LEAP’s semantics.

6.3. Relationship to the technical LEAP study

The technical LEAP study constructs the source-linked record and publication artifact. At result level, that companion already reports the Yield source and non-positive guard, the Gross Margin declared/executable denominator disagreement and *Needs Review* state, the existence of the 28-dossier build, and the publication architecture. None of those observations is reclaimed here. The present paper adds the executable four-obligation audit, complete-corpus lineage/guard/authority/conflict classification, reduction to three generator defect classes with root-cause localization, the formal positive-domain Gross Margin bound, within-LEAP persistence/input-sensitivity analysis, checker-validation controls, and the external FastAPI/OpenAPI transfer pilot. All evidence needed for those reported audit outcomes is included in the supplement; acceptance of the companion manuscript is not required to reproduce the analysis.

7. Threats to Validity

Construct validity. The four obligations are a literature-grounded checklist, not a complete or minimal theory of explanation quality. The LEAP condition rule is intentionally restricted to explicit zero/non-positive denominator safeguards. In the FastAPI pilot, condition disclosure is operationalized as publication of declarative validation constraints;

that adaptation is reported explicitly rather than treated as identical to arbitrary branch disclosure.

Internal validity. LEAP classifications and the external pilot are produced by deterministic scripts written for this study. Determinism does not by itself establish that a rule is conceptually correct. Negative-control testing exposed two false-pass behaviors in the earlier checker implementation; the reported results use the repaired implementation. The 30 binary checker controls cover specified edge cases with zero observed misses or false alarms, but those controls are authored tests rather than an independent population sample and therefore do not establish a general sensitivity/specificity estimate. To make inspection possible, the supplement retains the validation fixtures and mutations, line-level outputs, fixed invariants, generator/source hashes, external source identities, generated OpenAPI descriptions, and runtime validation outcomes.

Within-system persistence validity. The current and earlier LEAP builds are different archived project states, not independent systems. The identical lineage row arises because the same four dossiers carry the same 12 manifestations in both builds; the same six conflict cases are also shared. These observations are reported as persistence of named cases, not independent replication. The two additional LEAP configurations test input sensitivity only.

External validity. The FastAPI pilot supplies an independently developed generator and independently authored source corpus, but it is small and purposively selected to exercise auditable publication features. Seven direct checks pass; nine checklist cells are N/A. The pilot therefore demonstrates procedural transfer for source-role and declarative-condition checks, not defect prevalence and not full external validation of the disagreement or review-authority obligations. Broader cross-system studies should include publication systems that expose explicit competing representations and review-state metadata.

Prior-art boundary. ECK and Reversa are preprints rather than peer-reviewed publications and are treated as adjacent emerging work, not as authoritative validation of this study. ECK already evaluates projection fidelity, and Zhou et al. already detect API documentation defects involving parameter constraints and exception directives; neither capability is claimed as new here. The contribution boundary is grounded primarily in peer-reviewed documentation, inconsistency, evidence, API-directive, and provenance research together with the W3C PROV and OpenAPI specifications.

8. Conclusion

A source-linked publication can preserve evidence while communicating that evidence incorrectly. The complete current LEAP audit localizes three generator defects—lineage typing, guard-to-safeguard binding, and unconditional certification language—and one correctly operating conflict-routing mechanism. An earlier LEAP build shows persistence of the same four lineage-affected dossiers and the same six conflict-routed dossiers; alternative LEAP configurations

reproduce the lineage and authority defects but not the guard-binding defect. These are within-system persistence and input-sensitivity observations, not independent replication or prevalence estimates.

A separate external pilot applies the repaired checklist to four official FastAPI examples and generated OpenAPI descriptions. Before that rerun, 32 controlled checker outcomes match their expected labels with zero observed misses or false alarms in the specified controls. All seven directly applicable external source-role and condition checks then pass, while nine cells correctly return N/A; two deprecation-status analogues also pass. This establishes a limited transferability result for the validated role/condition procedure, while system-specific semantics remain explicit rather than being forced into a universal taxonomy.

The resulting contribution is therefore a reproducible publication-audit procedure with documented adaptation boundaries, together with defect localization in LEAP. Existing research supplies the underlying concepts; the paper contributes their operational use as inspectable checks at the boundary where software evidence becomes reader-facing publication.

Data and artifact availability

The supplementary archive contains the LEAP rendered corpora, archive and page hashes, the shared audit-rule library, checker-validation fixtures and actual-artifact mutations, deterministic audit scripts and outputs, the current-build guard inventory, alternative-configuration results, root-cause generator excerpts, and the detailed Yield and Gross Margin sources. It also contains the four official FastAPI tutorial source snapshots, verified upstream blob identities, generated OpenAPI descriptions, external audit tables, negative controls, and runtime validation outcomes. These materials reproduce every result reported in the paper.

CRedit authorship contribution statement

Meshaal Obaid Mouawad: Conceptualization, Methodology, Formal analysis, Investigation, Data curation, Visualization, Writing – original draft.

Declaration of competing interest

The author declares that there are no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During preparation of this work, the author used Elsevier LeapSpace to support literature discovery, manuscript organization, language refinement, and consistency checking. The author reviewed and edited the resulting material and takes full responsibility for the content of the article.

References

- Aghajani, E., Nagy, C., Vega-Márquez, O.L., Linares-Vásquez, M., Moreno, L., Bavota, G., Lanza, M., 2019. Software documentation issues unveiled, in: Proceedings of the 41st International Conference on Software Engineering, pp. 1199–1210. doi:[10.1109/ICSE.2019.00122](https://doi.org/10.1109/ICSE.2019.00122).
- Brush, M.H., Shefchek, K., Haendel, M., 2016. SEPIO: A semantic model for the integration and analysis of scientific evidence. CEUR Workshop Proceedings, Vol. 1747, International Conference on Biomedical Ontology. URL: https://ceur-ws.org/Vol-1747/IT605_ICB02016.pdf.
- Diamantini, C., Khan, T., Potena, D., Storti, E., 2025. Semantic models of performance indicators: A systematic survey. ACM Computing Surveys 57. doi:[10.1145/3719291](https://doi.org/10.1145/3719291).
- Domínguez, E., Pérez, B., Rubio, Á.L., Zapata, M.A., 2025. A flexible framework to ensure traceability, consistency, and propagation of KPIs evolution. Journal of Software: Evolution and Process 37, e70004. doi:[10.1002/smr.70004](https://doi.org/10.1002/smr.70004).
- Eghbali, A., Liu, Z., Pradel, M., 2026. Natural language-focused software engineering via code-documentation equivalence. Proceedings of the ACM on Software Engineering 3, 4644–4665. doi:[10.1145/3808211](https://doi.org/10.1145/3808211).
- FastAPI Project, 2026. FastAPI tutorial source examples. GitHub repository, commit 33d411dbc3236275dd64d200bfe18d5d60a49b2e. URL: https://github.com/fastapi/fastapi/tree/33d411dbc3236275dd64d200bfe18d5d60a49b2e/docs_src.
- Gao, X., 2026. Executable code knowledge: Code as a native, validation-carrying knowledge representation for AI coding agents. arXiv preprint arXiv:2608.16295 URL: <https://arxiv.org/abs/2608.16295>.
- Giglio, M., Tauber, R., Nadendla, S., Munro, J., Olley, D., Ball, S., Mitraka, E., Schriml, L.M., Gaudet, P., Hobbs, E.T., Erill, I., Siegele, D.A., Hu, J.C., Mungall, C., Chibucos, M.C., 2019. ECO, the evidence & conclusion ontology: community standard for evidence information. Nucleic Acids Research 47, D1186–D1194. doi:[10.1093/nar/gky1036](https://doi.org/10.1093/nar/gky1036).
- de Macedo, S.O., da Costa, R.M., 2026. Reversa: A reverse documentation engineering framework for converting legacy software into operational specifications for AI agents. arXiv preprint arXiv:2605.18684 URL: <https://arxiv.org/abs/2605.18684>.
- Mäder, P., Gotel, O., 2012. Towards automated traceability maintenance. Journal of Systems and Software 85, 2205–2227. doi:[10.1016/j.jss.2011.10.023](https://doi.org/10.1016/j.jss.2011.10.023).
- Maté, A., Trujillo, J.C., Mylopoulos, J., 2017. Specification and derivation of key performance indicators for business analytics: A semantic approach. Data & Knowledge Engineering 108, 30–49. doi:[10.1016/j.datak.2016.12.004](https://doi.org/10.1016/j.datak.2016.12.004).
- Missier, P., Belhajjame, K., Cheney, J., 2013. The W3C PROV family of specifications for modelling provenance metadata, in: Proceedings of the 16th International Conference on Extending Database Technology, pp. 773–776. doi:[10.1145/2452376.2452478](https://doi.org/10.1145/2452376.2452478).
- Nuseibeh, B., Easterbrook, S., Russo, A., 2001. Making inconsistency respectable in software development. Journal of Systems and Software 58, 171–180. doi:[10.1016/S0164-1212\(01\)00036-X](https://doi.org/10.1016/S0164-1212(01)00036-X).
- OpenAPI Initiative, 2026. OpenAPI specification, version 3.1. OpenAPI Initiative specification. URL: <https://spec.openapis.org/oas/v3.1>.
- Psallidas, F., Agrawal, A., Sugunan, C., Ibrahim, K., Karanasos, K., Camacho-Rodríguez, J., Floratou, A., Curino, C., Ramakrishnan, R., 2023. OneProvenance: Efficient extraction of dynamic coarse-grained provenance from database query event logs. Proceedings of the VLDB Endowment 16, 3662–3675. doi:[10.14778/3611540.3611555](https://doi.org/10.14778/3611540.3611555).
- Roldán-García, M.d.M., García-Nieto, J.M., Maté, A., Trujillo, J.C., Aldana-Montes, J.F., 2021. Ontology-driven approach for KPI meta-modelling, selection and reasoning. International Journal of Information Management 58, 102018. doi:[10.1016/j.ijinfomgt.2019.10.003](https://doi.org/10.1016/j.ijinfomgt.2019.10.003).
- Sadiq, M.A., Langat, P.K., Neupane, A., 2026. Enterprise spatial data provenance knowledge infrastructure. ISPRS International Journal of Geo-Information 15, 182. doi:[10.3390/ijgi15050182](https://doi.org/10.3390/ijgi15050182).
- Wen, F., Nagy, C., Bavota, G., Lanza, M., 2019. A large-scale empirical study on code-comment inconsistencies, in: Proceedings of the 27th International Conference on Program Comprehension, pp. 53–64. doi:[10.1109/ICPC.2019.00019](https://doi.org/10.1109/ICPC.2019.00019).
- World Wide Web Consortium, 2013. Constraints of the PROV data model. W3C Recommendation. URL: <https://www.w3.org/TR/prov-constraints/>.
- Xu, Z., Guo, S., Wang, Y., Chen, R., Li, H., Li, X., Jiang, H., 2024. Code comment inconsistency detection based on confidence learning. IEEE Transactions on Software Engineering 50, 598–617. doi:[10.1109/TSE.2024.3358489](https://doi.org/10.1109/TSE.2024.3358489).
- Zhou, Y., Gu, R., Chen, T., Huang, Z., Panichella, S., Gall, H.C., 2017. Analyzing APIs documentation and code to detect directive defects, in: Proceedings of the 39th IEEE/ACM International Conference on Software Engineering, pp. 27–37. doi:[10.1109/ICSE.2017.11](https://doi.org/10.1109/ICSE.2017.11).